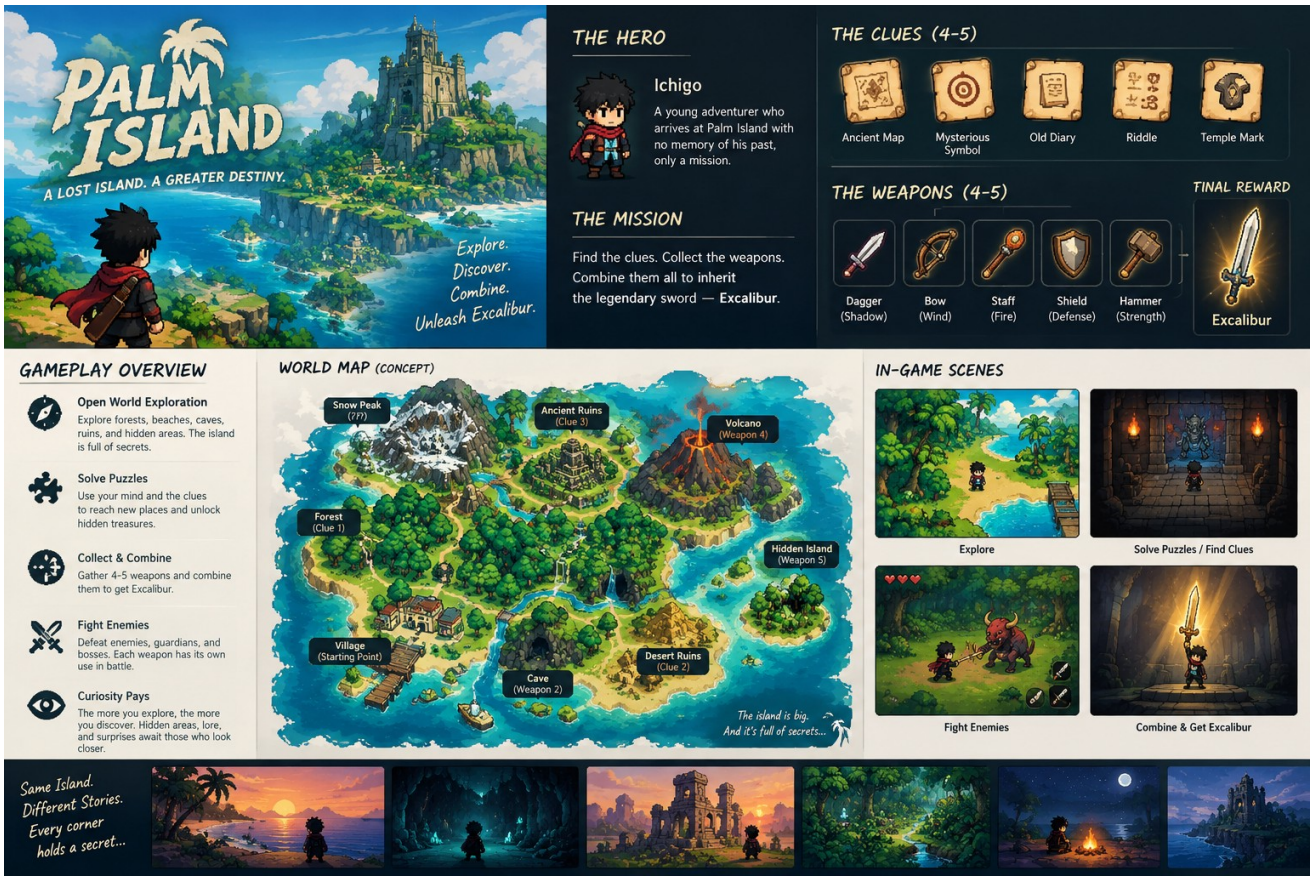
PALM ISLAND

GAME DESIGN DOCUMENT & DEVELOPMENT ROADMAP

Version 1.0 • Windows PC • C11 + Raylib • Anime-inspired 2D side-scrolling adventure



Tagline: One island. Five legendary weapons. One sword worthy of a hero.

1. Game Overview

Palm Island is a 2D side-scrolling open-world adventure starring Ichigo. The player explores a mysterious tropical island, discovers five clues, collects five legendary weapons, battles guardians, solves environmental puzzles, uncovers secrets, and earns the right to wield Excalibur.

The defining feature is curiosity: every unexplored path might conceal a story, secret, treasure, or route forward.

Category	Design
Genre	2D action-adventure
Camera	Side-scrolling
Art direction	Anime-inspired
Platform	Windows PC
Technology	C11 + Raylib
Mode	Single-player, offline

Design Pillars

- Curiosity over quest markers: reward observation and experimentation.
- Every weapon changes gameplay through combat and exploration abilities.
- Freedom with meaningful progression: regions are interconnected, with ability-gated routes.
- Combat with purpose: enemies provide challenge, resources, lore, and rewards.
- A memorable payoff: Excalibur must feel earned through discovery and mastery.

2. Story and Narrative

Ichigo arrives on Palm Island after discovering an old symbol associated with a forgotten kingdom. Ancient mechanisms still operate, guardians protect buried relics, and strange symbols appear in places where nobody should have visited for centuries.

An inscription describes five weapons that belonged to the island's legendary protectors. Legends claim these weapons can awaken Excalibur, a sword hidden in a sanctuary beneath the island.

A possible twist: Excalibur was never broken into five pieces. The five weapons were created to test the person worthy of wielding it. Each represents a quality: agility, strength, awareness, precision, and courage.

Narrative Delivery

- Ancient inscriptions reveal history in short fragments.
- Explorer journals provide optional stories and puzzle hints.
- NPC dialogue offers rumors, warnings, and local legends.
- Environmental storytelling uses abandoned camps, broken statues, sealed doors, and traces of previous explorers.
- Guardian encounters reveal something about each weapon.

3. The Five Clues and Legendary Weapons

1. Thornfang — The Jungle Dagger

Region: Forgotten Jungle. Clue: The Map of Roots, found in a shipwreck. Guardian: Thornback, a creature covered in vines and bark.

Combat: fast attacks, three-hit combo, low stamina cost. Exploration: cuts thorn barriers and supports a quick recovery slash after a dodge. Puzzle: follow markings on trees to find a concealed shrine.

2. Embercleaver — The Flame Axe

Region: Volcanic Caverns. Clue: The Broken Tablet. Guardian: Magma Golem, a slow enemy with powerful area attacks.

Combat: heavy strikes, armor breaking, charged ground slam. Exploration: smashes cracked stone walls and activates heavy pressure plates. Puzzle: redirect molten flow to power an ancient forge.

3. Stormpiercer — The Lightning Spear

Region: Sunken Temple. Clue: The Explorer's Journal. Guardian: Tide Warden, a protector that manipulates water and electricity.

Combat: long-reach thrusts and charged lightning strike. Exploration: activates electrical devices and conducts power through special crystal nodes. Puzzle: connect three power nodes in the correct sequence.

4. Dawnstring — The Sun Bow

Region: Skywatch Cliffs. Clue: The Shadow of the Sun. Guardian: Sunhawk, a flying enemy attacking from different elevations.

Combat: aimed shots, charged arrows, ranged attacks. Exploration: hits distant switches, cuts ropes, and redirects beams using special targets. Puzzle: align mirrors to illuminate an ancient seal.

5. Voidfang — The Shadow Blade

Region: Lost Kingdom. Clue: The King's Last Secret. Guardian: Hollow Knight, a skilled swordsman who mirrors Ichigo's attacks.

Combat: short-range shadow dash, counterattack, brief invulnerability during a special move. Exploration: crosses shadow gates and bypasses certain magical barriers. Puzzle: reconstruct a royal memory from five hidden symbols.

Weapon Design Rules

- Collected weapons remain permanently available; players can switch between them.
- Each weapon has a basic attack and at least one special move.
- Exploration abilities should have obvious visual feedback.
- Bosses should teach weapon use rather than require repeated attacks.
- Weapon switching should be fast and support creative combinations.

4. World Map and Progression

Build the island from smaller side-scrolling areas connected by paths, caves, gates, and shortcuts. This creates an open-world feeling without requiring one enormous level to be loaded at once.

Region	Purpose	Main unlock
Palmshore Beach	Tutorial and basic movement	Access to village
Driftwood Village	NPCs, shops, rumors	First clue
Forgotten Jungle	Exploration and first guardian	Thornfang
Ember Caverns	Heavy obstacles and forge puzzle	Embercleaver
Sunken Temple	Electrical mechanisms and water hazards	Stormpiercer
Skywatch Cliffs	Vertical platforming and ranged puzzles	Dawnstring
Lost Kingdom	Advanced combat and hidden passages	Voidfang
Excalibur Sanctuary	Final trials and ending	Excalibur

Suggested Connectivity

- Palmshore Beach → Driftwood Village → Forgotten Jungle.
- Forgotten Jungle branches toward Ember Caverns and Sunken Temple.
- Ember Caverns and Sunken Temple connect toward Skywatch Cliffs.
- Skywatch Cliffs → Lost Kingdom → Excalibur Sanctuary.
- Optional shortcuts connect later areas back to earlier regions, such as a secret passage to the village or an underwater route into the jungle.

Progression Gate Types

- Ability gates: thorn walls, cracked walls, and shadow barriers require specific weapons.
- Knowledge gates: riddles, symbol sequences, and clues require understanding.
- Skill gates: difficult platform sections and guardians test movement and combat.
- Not every locked door needs a key; sometimes the solution is hidden in the environment.

5. Player Controls and Movement

Action	Default input
Move	A / D or Left / Right
Jump	Space
Dodge / dash	Left Shift
Basic attack	J or left mouse button
Special attack	K or right mouse button
Switch weapon	1–5
Interact / talk	E
Open map	M
Inventory	I
Pause	Esc

Movement Mechanics

- Acceleration and deceleration instead of instant horizontal movement.
- Jumping with variable height.
- Coyote time: a short grace period after leaving a ledge.
- Jump buffering: a jump pressed just before landing is queued.
- Dodge with a short cooldown.
- Ladders, ledges, moving platforms, and one-way platforms.
- Limit or disable fall damage initially to avoid punishing exploration.

6. Combat and Enemy Design

Start with health and attack cooldowns. Add stamina or a special-energy meter only after movement and combat feel enjoyable.

Property	Prototype starting value
Maximum health	100 HP
Common enemy health	30–50 HP
Mini-boss health	180–250 HP
Main guardian health	350–500 HP
Basic sword damage	15 HP
Dodge duration	0.18 seconds
Basic attack cooldown	0.35 seconds
Respawn	Nearest checkpoint

Enemy Roster

- Jungle Slime: slow, low health, simple contact attack; teaches movement and dodging.
- Island Raider: approaches Ichigo, telegraphs a strike, and blocks careless attacks.
- Ruin Wraith: fires slow projectiles and forces the player to use cover, movement, or ranged weapons.
- Ancient Guardian: elite enemy with distinct attacks, clear wind-ups, and recovery windows.

Guardian Encounter Structure

1. Observe: the guardian demonstrates its attack pattern.
2. Adapt: the player learns when to dodge, counter, or use the environment.
3. Master: the guardian becomes more aggressive at low health without unfair attacks.

Use visual telegraphs such as glowing ground markers, weapon flashes, animation cues, and audio. Apply damage through hitboxes and hurtboxes rather than simply checking whether sprites overlap.

7. Puzzles, Secrets and Rewards

- Symbol puzzles: observe a clue, find matching symbols, and activate them in the correct order.
- Environmental puzzles: move objects, redirect beams, activate switches, or use weapon abilities.

- Optional secrets: hidden treasure rooms, rare upgrades, secret NPCs, lore pages, and shortcuts. Some secrets should be hinted at rather than marked on the map.

Reward Categories

- Coins for healing items and upgrades.
- Health upgrades for surviving harder areas.
- Weapon upgrades that modify damage or special effects.
- Lore fragments that deepen the story.
- Cosmetic rewards such as alternate outfits.
- Hidden challenges that reward exploration rather than grinding.

8. Technical Architecture: C + Raylib

Build a small custom 2D game engine on Raylib. Raylib handles window creation, rendering, keyboard/controller input, textures, audio, and collision primitives. Game code handles world simulation, physics, combat, enemies, puzzles, inventory, saving, and progression.

Technology	Recommendation
Language	C11
Framework	Raylib
Compiler	GCC via MinGW-w64 on Windows
Build system	CMake
Level editor	Tiled; TMX/TSX or a custom format
Data	JSON for definitions and save metadata
Images	PNG spritesheets and tilesets
Audio	WAV sound effects and OGG music
Version control	Git

Suggested Project Layout

- CMakeLists.txt — build configuration.
- README.md — setup and development notes.
- src/ — C source files.
- include/ — headers and shared interfaces.
- assets/ — sprites, tilesets, audio and fonts.
- data/ — levels, enemies, weapons and puzzles.
- saves/ — local save files during development; user saves should live in LocalAppData.
- tests/ — unit tests and logic tests.

Module Responsibilities

Module	Responsibility
main.c	Initialization, main loop, shutdown
game.c/.h	Global game state and high-level update flow
input.c/.h	Input mapping and control state
player.c/.h	Movement, jumping, health and animation
combat.c/.h	Attacks, hitboxes, damage and invulnerability

weapon.c/.h	Weapon definitions, switching and abilities
enemy.c/.h	Enemy behaviors and AI
boss.c/.h	Guardian phases and boss logic
world.c/.h	Areas, transitions, collision and loading
physics.c/.h	Gravity, collision resolution and movement
puzzle.c/.h	Switches, gates, mechanisms and puzzle states
interaction.c/.h	NPC dialogue, clues and interactable objects
camera.c/.h	Side-scrolling camera and boundaries
render.c/.h	World rendering, sprites and effects
ui.c/.h	HUD, menus, inventory and map
save.c/.h	Serialization, validation and checkpoints
audio.c/.h	Music and sound-effect management
assets.c/.h	Asset loading, caching and cleanup

Main Game Loop

Use Raylib's standard frame loop with a fixed timestep for gameplay simulation and normal rendering.

```
#include "raylib.h"

int main(void)
{
    InitWindow(1280, 720, "Palm Island");
    SetTargetFPS(60);

    while (!WindowShouldClose())
    {
        // Read input.
        // Update gameplay at a fixed timestep.
        // Update camera and animations.
        // Render world and UI.

        BeginDrawing();
        ClearBackground(BLACK);

        // RenderGame();

        EndDrawing();
    }

    // UnloadGameAssets();
    CloseWindow();
    return 0;
}
```

Core Programming Principles

- Use structs and enums to group related data.
- Keep weapon behaviors out of a giant switch in main.c; use common definitions and a weapon module.

- Separate simulation from rendering: UpdatePlayer() changes state, DrawPlayer() draws it.
- Use stable IDs for enemies, puzzles, and chests so their states can persist.
- Load and cache assets once rather than every frame or map visit.

Example data structure:

```
typedef enum {
    WEAPON_NONE,
    WEAPON_THORNFANG,
    WEAPON_EMBERCLEAVER,
    WEAPON_STORMPIERCER,
    WEAPON_DAWNSTRING,
    WEAPON_VOIDFANG,
    WEAPON_EXCALIBUR
} WeaponId;

typedef struct {
    Vector2 position;
    Vector2 velocity;
    float health;
    float maxHealth;
    int facing;         // -1 = left, 1 = right
    bool grounded;
    WeaponId equipped;
} Player;
```

9. World and Entity Representation

Level Layers

- background: distant mountains, clouds, and scenery.
- ground: floor and platform tiles.
- collision: solid walls and terrain.
- objects: chests, switches, gates, ladders, and hazards.
- entities: enemy and NPC spawn points.
- triggers: room transitions, secrets, checkpoints, and boss activation.

Each area is a separate map file, and the world system manages connections between them. This gives an open-world feel without loading every island asset at once.

Persistent World State

Give each object a stable ID, such as jungle_chest_03. Once opened, record its state in progression data. Reloading the map must not recreate its treasure. Gates should check the underlying progression state, and both their visual appearance and collision should update from that same state.

10. Save System

What to Save

- Player position, current area, health and equipped weapon.
- Five weapon ownership flags and weapon upgrades.
- Clues discovered and lore entries collected.
- Bosses defeated and puzzle states completed.
- Opened treasure chests and permanent world changes.
- Coins, inventory and player upgrades.
- Checkpoint position and main story progression.
- Save version and basic integrity metadata.

Illustrative JSON Save Structure

```
{
  "version": 1,
  "area": "forgotten_jungle",
  "player": {
    "x": 420.0,
    "y": 180.0,
    "health": 85,
    "equipped_weapon": 1
  },
  "weapons": {
    "thornfang": true,
    "embercleaver": false,
    "stormpiercer": false,
    "dawnstring": false,
    "voidfang": false
  },
  "progress": {
    "clues_found": 1,
    "guardians_defeated": 1,
    "excalibur_unlocked": false
  },
  "world": {
    "jungle_chest_03": true,
    "jungle_gate_01": false
  }
}
```

Saving Strategy

4. Save at checkpoints and after major milestones.
5. Allow manual saving at designated save points.
6. Write to a temporary file and replace the previous save only after a successful write.
7. Keep a backup of the last valid save.
8. Validate the version and ranges when loading.

9. If loading fails, offer the backup or a new game rather than crashing.

Store user saves in %LOCALAPPDATA%/PalmIsland/ rather than relying on the current working directory. Begin with one main save slot and one backup; add multiple slots later.

11. Development Roadmap

Phase	Estimate	Deliverable
1. Foundation	Week 1	CMake, Raylib, game loop, input, rendering and camera.
2. Player controller	Weeks 2–3	Movement, jumping, gravity, collision, animation and respawn.
3. Combat prototype	Weeks 4–5	Hitboxes, enemy AI, health, damage, dodge and one guardian.
4. Exploration prototype	Weeks 6–7	Tiled map, area transitions, clue, hidden chest, puzzle and Thornfang.
5. Persistence	Week 8	Inventory, world flags, checkpoints and save/load.
6. Vertical slice	Weeks 9–11	One polished region with clue, weapon, secret, enemies, guardian, dialogue and audio.
7. Content production	Weeks 12–20+	Remaining regions, weapons, guardians, clues, secrets and story.
8. Polish and release	Weeks 21–24+	Balance, effects, audio, optimization, bug fixes and Windows packaging.

These estimates assume a part-time solo developer. A complete indie release may take around six months or longer, depending on asset creation and scope.

12. First Playable Milestone: The Forgotten Jungle

Target a self-contained 10–15 minute experience. It should prove that movement, combat, exploration, a clue, a secret, a weapon ability, and saving work together.

- One Ichigo character with idle, run, jump, fall, attack, hurt, and death states.
- One small side-scrolling jungle level with solid terrain, platforms, camera tracking, and checkpoint.
- One basic sword attack and dodge.
- Two enemy types.
- One hidden chest and one environmental puzzle.
- The first clue and Thornfang weapon.
- One guardian boss with at least two attack patterns.

- A simple HUD showing health and equipped weapon.
- Save/load for weapon, clue, and boss progression.

Definition of Done

- Start a new game.
- Explore without falling through terrain or becoming stuck.
- Find the clue and understand its hint.
- Discover a hidden secret.
- Fight ordinary enemies and defeat the guardian.
- Collect Thornfang and use its ability to reach a previously inaccessible area.
- Save, quit, relaunch, and resume without losing progression.
- Reach the end without a crash or game-breaking bug.

13. Development Risks and Priorities

Risk	Mitigation
Scope becomes too large	Build one polished region first; expand only after testing it.
Combat feels unresponsive	Tune movement, hitboxes, attack timing, and animation before adding content.
Open-world levels become hard to manage	Split the island into interconnected maps with shared world-state data.
Save files become inconsistent	Use versioned saves, stable object IDs, validation, and backups.
Asset creation slows progress	Establish sprite dimensions and animation conventions early.
New weapons require rewrites	Use shared weapon interfaces and data-driven definitions.
Too many secrets confuse players	Provide environmental hints and optional journal clues.

Minimum Viable Release vs. Later Features

First full release: five clues, five weapons, five major guardians, interconnected regions, combat, puzzles, inventory, save/load, and the Excalibur finale.

Later updates: multiple endings, large side-quest chains, crafting, complex equipment, dynamic weather/day-night cycles, extra playable characters, and procedural generation.

14. Testing Checklist

- Movement and collisions at different frame rates.
- Attacks hit enemies only once per intended attack window.
- Weapon switching during combat.
- Guardian death, reward collection, and checkpoint behavior.
- Puzzles remain solved after loading a save.

- Permanent collectibles do not respawn.
- Missing or corrupted save files are handled safely.
- Transitions between connected regions work.
- Textures and maps are unloaded correctly.
- Windows builds run without development-only files or absolute asset paths.

Use Git from day one, commit working milestones, and maintain a known-good build.

15. First Week Implementation Checklist

Task	Completion criteria
Install Raylib and a C compiler	A minimal Raylib program compiles on Windows.
Create the CMake project	A repeatable Debug executable builds from the project root.
Open a 1280 × 720 window	Window displays a title and closes cleanly.
Draw temporary Ichigo sprite	Use a placeholder rectangle or sprite before final art.
Implement movement and jumping	Gravity, ground collision, and a platform work.
Implement side-scrolling camera	Camera follows Ichigo and respects level boundaries.
Commit to Git	Clean baseline exists before combat is added.

Final Recommendation

Build Palm Island as a sequence of increasingly complete experiences: (1) movement prototype, (2) combat prototype, (3) exploration prototype, (4) vertical slice, and (5) full game. Keep code modular, world state persistent, and each weapon's combat and exploration abilities independent.

The first concrete coding target is a controllable Ichigo moving through a single jungle platforming level in Raylib. Once that works smoothly, every other system has a foundation to build on.